

Liqua Monitor – Descrição do Backend, API e Infraestrutura

Desenvolvedores Backend: Davi Coelho Maciel e Pedro Domiense dos Santos

Autor: Davi Coelho Maciel

Turma: 3º ano Desenvolvimento de Sistemas

Sumário

1. Introdução	2
2. Arquitetura	2
3. Principais Tecnologias utilizadas	4
4. Modo de execução	4
5. Subdivisões e endpoints da API	5
5.1. Usuários	5
5.1.1. /api/usuarios/criar_usuario	5
5.1.2. /api/usuarios/info_usuario_logado	5
5.1.3. /api/usuarios/editar_usuario	6
5.1.4. /api/usuarios/deletar_usuario	6
5.1.5. /api/usuarios/login_usuario	6
5.1.6. /api/usuarios/logout_usuario	7
5.1.7. /api/usuarios/regerar_token	7
5.1.8. /api/usuarios/esqueci_a_senha	8
5.1.9. /api/usuarios/recuperar_senha	8
5.2. Consumos Reais	8
5.2.1. /api/consumos/criar_consumo	8
5.2.2. /api/consumos/listar_consumos	9
5.2.3. /api/consumos/editar_consumo/{id}	9
5.2.4. /api/consumos/deletar_consumo/{id}	10
5.3. Simulações de Consumo	10
5.3.1. /api/simulacoes/criar_simulacao	10
5.3.2. /api/simulacoes/listar_simulacoes	10
5.3.3. /api/simulacoes/editar_simulacao/{id}	11
5.3.4. /api/simulacoes/deletar_simulacao/{id}	11
5.4. Metas de Consumo	12
5.4.1. /api/metas/criar_meta	12
5.4.2. /api/metas/listar_metas	12
5.4.3. /api/metas/editar_meta/{id}	13
5.4.4. /api/metas/deletar_meta/{id}	13

5.5. Dicas Sustentáveis	13
5.5.1. /api/dicas/pegar_dica_aleatoria/{unit}.....	13
6. Infraestrutura	14
7. Considerações Finais	14

1. Introdução

Esse sistema foi desenvolvido com o objetivo de estruturar a lógica de negócio na forma de um *backend* e expor na internet uma conexão com o banco de dados, em forma de API, para o aplicativo mobile Liqua Monitor, um gerenciador e monitor de consumos domésticos com espaço de usuário dedicado.

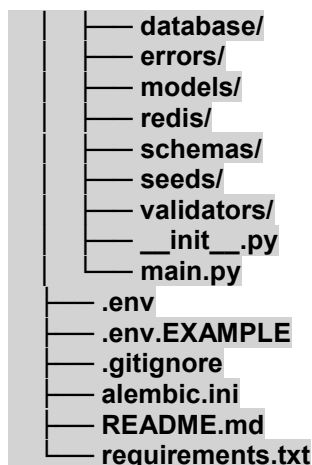
Inicialmente se tratando apenas de operações CRUD (criar, ler, atualizar e excluir), rapidamente o sistema evoluiu para um fluxo completo de usuário preparado para receber o *frontend*, com segurança nas transmissões de dados, exposição via HTTPS da API, validação e normalização de dados. Além disso, o foco original em desenvolver as operações voltadas ao gerenciamento de consumos possui funções essenciais com segurança de banco de dados, fluxos amigáveis e pouco exigentes aos usuários e modularidade essencial para manutenção adequada no time de desenvolvimento.

2. Arquitetura

A arquitetura foi inspirada no popular padrão de design de software MVC (models-views-controllers), muito utilizada em aplicações fechadas em Python e originada no Rails. No entanto, como a exposição do sistema é feita na forma de API e não telas, o fluxo principal foi adaptado para MAC (models-api-controllers), também um design comum devido à divisão de responsabilidades. As outras camadas de código-fonte foram implementadas de forma altamente modularizada, reduzindo acoplamento e separando responsabilidades lógicas, para que funções grandes fossem reduzidas, chamadas internas fossem padronizadas e a manutenção fosse facilitada.

Em estágios finais de desenvolvimento, a arquitetura de pastas em ambiente IDE (Ambiente Integrado de Desenvolvimento), essa era a árvore de código-fonte:





Para reforçar a modularidade, a lógica de API ainda foi subdividida na pasta responsável pela exposição dos *endpoints* e em outra, denominada *auth/*, que trata da criação e manipulação dos *tokens* JWT necessários para o fluxo de usuário.

A divisão de responsabilidades dentro do código-fonte (*src/*) é a seguinte:

- *api/*: Endpoints FastAPI que realizam a conexão entre a aplicação e a web utilizando dados padronizados em *requests* e respostas.
- *auth/*: Instancia e gerencia o estado dos *tokens* JWT do sistema, permitindo autenticação segura.
- *controllers/*: Onde as lógicas de negócio são determinadas e a manipulação do banco de dados é feita de acordo com as requisições.
- *database/*: Onde a conexão com o banco de dados principal é feita e as sessões, formas com que os usuários interagem com o sistema, são gerenciadas dinamicamente.
- *errors/*: Erros internos do sistema, criados tanto para resolver os códigos de erro HTTP corretos quanto para indicar falhas nas diferentes camadas do software.
- *models/*: Modelos SQLAlchemy que permitem a conexão entre *backend* e banco de dados, além da manipulação de dados por meio de *queries* montadas na própria linguagem de programação.
- *redis/*: Conexão do *redis-py*, biblioteca para conexão Redis no Python, que permite a conexão entre a infraestrutura do Redis e o sistema. Diferencia-se da *redis/* em *infra/* pela responsabilidade, que aqui é inteiramente referente ao Python.
- *seeds/*:
- *schemas/*: Os *schemas* do Pydantic utilizados para validação de dados e padronização de respostas na API, exigindo tipos específicos e uma estrutura predeterminada na forma de JSON para entradas e saídas de dados do sistema.
- *validators/*: Scripts para validação independente de dados que chegam ao *backend*, podendo recusar a persistência de *usernames*, e-mails e senhas cadastradas que não estejam de acordo com as regras exigidas.

- main.py: O arquivo de execução da API FastAPI, único ponto de entrada do *runtime* e arquivo que é executado para o *backend* ir ao ar.

E dentro da pasta de infraestrutura e devops (infra/), dessa forma:

- caddy/: A pasta onde se encontra o arquivo de configuração do Caddy, o web-server e proxy reverso utilizado para exposição do DNS da aplicação (<https://liquamonitor.dev/>) de forma segura, com HTTPS automático e configurações modulares que reduzem downtime.

- docker/: A pasta onde estão os arquivos de configuração do Docker, a tecnologia de containeres utilizada para componentizar e separar responsabilidades dentre os subsistemas que utilizamos em conjunto ao backend, criando um ambiente de execução próprio para cada um.

- redis/: A pasta onde está a configuração do Redis, o banco de dados *in-memory* e sistema de caching utilizado no sistema para invalidar tokens JWT de usuários que realizam log-out e de requisições de recuperação de senha já sucedidas.

3. Principais Tecnologias utilizadas

Sistema Gerenciador de Banco de Dados: PostgreSQL 17.9;

Banco de dados *in-memory* para lógica de *token*: Redis 8.6.2;

Linguagem de programação: Python 3.13.13;

ORM (Object-Relational Mapper): SQLAlchemy 2.0.46;

Validador de dados: Pydantic 2.12.5;

Construção e exposição da API: FastAPI 0.128.2;

Token seguro: JSON Web Token, de acordo com a RFC 7519;

Web-server e reverse-proxy: Caddy 2.11.2;

Ambiente de Desenvolvimento: Microsoft Visual Studio Code (versões diversas).

4. Modo de execução

A execução do código-fonte inteiro, clonado a partir do repositório git já com web-server configurado para produção e todas as tecnologias conectadas, deve ser realizada em um Virtual Private Server (VPS, servidor privado virtual) com o Docker sendo o único pré-requisito de software. Ademais, os arquivos “.env” e “/infra/redis/redis.conf” devem ser criados de acordo com os exemplos, fornecendo os dados corretos para a execução, e o domínio declarado na “/infra/caddy/Caddyfile” precisa ser substituído pelo domínio na posse do usuário.

Após isso, na pasta raiz do projeto, o seguinte comando deve ser executado:

```
docker compose -f ./infra/docker/compose.yaml up --build
```

Após isso, o sistema estará rodando na Docker Engine do servidor e já estará acessível.

5. Subdivisões e *endpoints* da API

A API exposta na web possui seis grandes divisões, todas dentro do caminho `/api/`: Usuários, Consumos, Simulações, Metas e Dicas, cada uma com *endpoints* dedicados às funções que detém no sistema. Cada um deles possui uma resposta padronizada e um corpo de requisição específico, ambos necessários para previsibilidade no *frontend*. A seguir, o comportamento esperado de todos eles.

5.1. Usuários

5.1.1. `/api/usuarios/criar_usuario`

Método: POST

Descrição: Cadastra um usuário no banco de dados do sistema.

Corpo de requisição:

```
{
  "real_name": "string",
  "username": "string",
  "email": "user@example.com",
  "password": "*****" #SecretStr
}
```

Resposta:

```
{
  "id": 0,
  "real_name": "string",
  "username": "string",
  "email": "user@example.com",
  "created_at": "2026-04-24T11:16:54.989Z"
}
```

5.1.2. `/api/usuarios/info_usuario_logado`

Método: GET

Descrição: Retorna as informações do usuário corrente no sistema com base nas informações de autenticação provenientes da header HTTP da página, adicionadas no ato de login.

Corpo de requisição: Sem parâmetros.

Resposta:

```
{
  "id": 0,
  "real_name": "string",
  "username": "string",
  "email": "user@example.com",
}
```

```
}
  "created_at": "2026-04-24T11:25:06.070Z"
}
```

5.1.3. /api/usuarios/editar_usuario

Método: PATCH

Descrição: Edita as informações do usuário com base nos parâmetros passados. Irá ignorar os parâmetros nulos no *payload*.

Corpo de Requisição:

```
{
  "new_real_name": "string", # Opcional
  "new_username": "string", # Opcional
  "new_email": "user@example.com", # Opcional
  "new_password": "*****" # Opcional
}
```

Resposta:

```
{
  "id": 0,
  "real_name": "string",
  "username": "string",
  "email": "user@example.com",
  "created_at": "2026-04-24T11:25:54.876Z"
}
```

5.1.4. /api/usuarios/deletar_usuario

Método: DELETE

Descrição: Exclui um usuário do banco de dados. Exige envio do token de *refresh* JWT associado a ele, que deve ser armazenado no *frontend*, para invalidação.

Corpo de Requisição:

```
{
  "refresh_token": "string"
}
```

Resposta: Sem resposta (204).

5.1.5. /api/usuarios/login_usuario

Método: POST

Descrição: Recebe e-mail e senha do usuário em um *form* padronizado OAuth2PasswordRequestForm, apesar do parâmetro de e-mail se chamar “username”. Responde com o token de acesso associado automaticamente à *header* “Authentication” do HTTP e o token de *refresh*, que o *frontend* deve armazenar pois é o responsável por manter viva a sessão do usuário.

Corpo de Requisição:

```
{
  "username": " user@example.com ", #Insira email, não username. Padrão OAuth.
  "password": "*****",
}
```

Resposta:

```
{
  "access_token": "string",
  "refresh_token": "string",
  "token_type": "string"
}
```

5.1.6. /api/usuarios/logout_usuario

Método: POST

Descrição: Realiza a invalidação de todos os tokens ativos do usuário, exigindo seu login novamente para instanciar novos tokens. Exige apenas o de *refresh*, pois o de acesso é pego automaticamente com base na *header*.

Corpo de Requisição:

```
{
  "refresh_token": "string"
}
```

Resposta: Sem resposta (204).

5.1.7. /api/usuarios/regerar_token

Método: POST

Descrição: Utiliza o token de *refresh* para gerar outro token de acesso, permitindo que o usuário continue a utilizar o sistema. O token retornado é automaticamente adicionado à *header*.

Corpo de Requisição:

```
{
  "refresh_token": "string"
}
```

Resposta:

```
{
  "access_token": "string",
  "token_type": "string"
}
```

5.1.8. /api/usuarios/esqueci_a_senha

Método: POST

Descrição: Acessado quando o usuário deseja recuperar sua senha, precisa do e-mail para tentar encontra-lo de forma segura. Não retorna nada via HTTP, porém enviará uma mensagem para o e-mail especificado no *payload* com um link para a recuperação de sua senha. Esse e-mail automaticamente instanciará um token de recuperação como *query parameter* no botão, necessário para a recuperação de senha.

Corpo de Requisição:

```
{
  "email": "user@example.com"
}
```

Resposta: Sem resposta (204).

5.1.9. /api/usuarios/recuperar_senha

Método: POST

Descrição: Acessado a partir do botão de recuperação de senha enviado no email, exige o token de recuperação que estará na url do botão como *query parameter* (?recovery_token=). Além disso, recebe nova a senha que o usuário deseja cadastrar.

Corpo de Requisição:

```
{
  "new_password": "*****",
  "recovery_token": "string"
}
```

Resposta:

```
{
  "id": 0,
  "real_name": "string",
  "username": "string",
  "email": "user@example.com",
  "created_at": "2026-04-24T11:34:26.845Z"
}
```

5.2. Consumos Reais

5.2.1. /api/consumos/criar_consumo

Método: POST

Descrição: Cadastra um novo consumo real ao histórico do usuário.

Corpo de Requisição:

```
{
  "starting_date": "2026-04-24",
  "ending_date": "2026-04-24",
  "si_measurement_unit": "string",
}
```



```
"value": 0.0
}
```

Resposta:

```
{
  "id": 0,
  "starting_date": "2026-04-24",
  "ending_date": "2026-04-24",
  "measurement_unit": "string",
  "value": 0.0
}
```

5.2.2. /api/consumos/listar_consumos

Método: GET

Descrição: Lista os consumos reais associados ao usuário corrente de acordo com os parâmetros especificados, que utiliza como filtros. Irá ignorar as informações marcadas como nulas no *payload*.

Corpo de Requisição:

```
{
  "measurement_unit": "string", # Opcional
  "starting_date": "2026-04-24", # Opcional
  "ending_date": "2026-04-24", # Opcional
  "minimum_value": 0.0, # Opcional
  "maximum_value": 0.0 # Opcional
}
```

Resposta:

```
[
  {
    "id": 0,
    "starting_date": "2026-04-24",
    "ending_date": "2026-04-24",
    "measurement_unit": "string",
    "value": 0.0
  }
]
```

5.2.3. /api/consumos/editar_consumo/{id}

Método: PATCH

Descrição: Edita as informações do consumo real especificado como parâmetro de *path*, alterando apenas as informações não-nulas no *payload*.

Corpo de Requisição: ID do consumo requisitado (path);

```
{
  "new_starting_date": "2026-04-24", # Opcional
  "new_ending_date": "2026-04-24", # Opcional
  "new_si_measurement_unit": "string", # Opcional
  "new_value": 0.0 # Opcional
}
```

```
}
```

Resposta:

```
{
  "id": 0,
  "starting_date": "2026-04-24",
  "ending_date": "2026-04-24",
  "measurement_unit": "string",
  "value": 0.0
}
```

5.2.4. /api/consumos/deletar_consumo/{id}

Método: DELETE

Descrição: Exclui o consumo especificado com base em seu id, que é passado como parâmetro de *path*.

Corpo de Requisição: ID do consumo requisitado (path).

Resposta: Sem resposta (204).

5.3. Simulações de Consumo

5.3.1. /api/simulacoes/criar_simulacao

Método: POST

Descrição: Cadastra uma nova simulação de consumo associada ao usuário.

Corpo de Requisição:

```
{
  "starting_date": "2026-04-24",
  "ending_date": "2026-04-24",
  "si_measurement_unit": "string",
  "value": 0.0
}
```

Resposta:

```
{
  "id": 0,
  "starting_date": "2026-04-24",
  "ending_date": "2026-04-24",
  "measurement_unit": "string",
  "value": 0.0
}
```

5.3.2. /api/simulacoes/listar_simulacoes

Método: GET

Descrição: Lista as simulações associadas ao usuário corrente de acordo com os parâmetros especificados, que utiliza como filtros. Irá ignorar as informações marcadas como nulas no *payload*.

Corpo de Requisição:

```
{
  "measurement_unit": "string", # Opcional
  "starting_date": "2026-04-24", # Opcional
  "ending_date": "2026-04-24", # Opcional
  "minimum_value": 0.0, # Opcional
  "maximum_value": 0.0 # Opcional
}
```

Resposta:

```
[
  {
    "id": 0,
    "starting_date": "2026-04-24",
    "ending_date": "2026-04-24",
    "measurement_unit": "string",
    "value": 0.0
  }
]
```

5.3.3. /api/simulacoes/editar_simulacao/{id}

Método: PATCH

Descrição: Edita as informações da simulação especificada como parâmetro de *path*, alterando apenas as informações não-nulas no *payload*.

Corpo de Requisição: ID do consumo requisitado (path);

```
{
  "new_starting_date": "2026-04-24", # Opcional
  "new_ending_date": "2026-04-24", # Opcional
  "new_si measurement unit": "string", # Opcional
  "new_value": 0.0 # Opcional
}
```

Resposta:

```
{
  "id": 0,
  "starting_date": "2026-04-24",
  "ending_date": "2026-04-24",
  "measurement_unit": "string",
  "value": 0.0
}
```

5.3.4. /api/simulacoes/deletar_simulacao/{id}

Método: DELETE

Descrição: Exclui a simulação especificada com base em seu id, que é passado como parâmetro de *path*.

Corpo de Requisição: ID do consumo requisitado (*path*).

Resposta: Sem resposta (204).

5.4. Metas de Consumo

5.4.1. /api/metast/criar_meta

Método: POST

Descrição: Cadastra uma nova meta de consumo associada ao usuário.

Corpo de Requisição:

```
{
  "starting_date": "2026-04-24",
  "ending_date": "2026-04-24",
  "si_measurement_unit": "string",
  "value": 0.0
}
```

Resposta:

```
{
  "id": 0,
  "starting_date": "2026-04-24",
  "ending_date": "2026-04-24",
  "measurement_unit": "string",
  "value": 0.0
}
```

5.4.2. /api/metast/listar_metast

Método: GET

Descrição: Lista as metas de consumo associadas ao usuário corrente de acordo com os parâmetros especificados, que utiliza como filtros. Irá ignorar as informações marcadas como nulas no *payload*.

Corpo de Requisição:

```
{
  "measurement_unit": "string", # Opcional
  "starting_date": "2026-04-24", # Opcional
  "ending_date": "2026-04-24", # Opcional
  "minimum_value": 0.0, # Opcional
  "maximum_value": 0.0 # Opcional
}
```

Resposta:

```
[
```

```
{
  "id": 0,
  "starting_date": "2026-04-24",
  "ending_date": "2026-04-24",
  "measurement_unit": "string",
  "value": 0.0
}
```

5.4.3. /api/metas/editar_meta/{id}

Método: PATCH

Descrição: Edita as informações da meta de consumo especificada como parâmetro de *path*, alterando apenas as informações não-nulas no *payload*.

Corpo de Requisição: ID do consumo requisitado (path);

```
{
  "new_starting_date": "2026-04-24", # Opcional
  "new_ending_date": "2026-04-24", # Opcional
  "new_si_measurement_unit": "string", # Opcional
  "new_value": 0 # Opcional
}
```

Resposta:

```
{
  "id": 0,
  "starting_date": "2026-04-24",
  "ending_date": "2026-04-24",
  "measurement_unit": "string",
  "value": 0.0
}
```

5.4.4. /api/metas/deletar_meta/{id}

Método: DELETE

Descrição: Exclui a meta de consumo especificada com base em seu id, que é passado como parâmetro de *path*.

Corpo de Requisição: ID do consumo requisitado (path).

Resposta: Sem resposta (204).

5.5. Dicas Sustentáveis

5.5.1. /api/dicas/pegar_dica_aleatoria/{unit}

Método: GET

Descrição: Utiliza a unidade de medida requerida via *path* como parâmetro para retornar uma dica sustentável aleatória do banco de dados, instanciada por meio do script de seeding

Corpo de Requisição: Unidade de medida desejada (path).

Resposta:

```
{
  "si_measurement_unit": "string",
  "tip": "string"
}
```

6. Infraestrutura

O sistema está hospedado na internet a partir do *web-server* e *reverse proxy* Caddy, um software integrado ao Docker que possui configuração fácil por meio de seu arquivo de configuração Caddyfile e o diferencial de gerenciar automaticamente os certificados HTTPS. Os domínios configurados são:

- liquamonitor.dev/: O domínio principal do sistema e onde a API está hospedada em /api/, com seguranças e proteção contra DDoS da Cloudflare. Possui headers completas, redirect automático para /docs em requisições que não são da API e bloqueio contra indexação em navegadores.
- www.liquamonitor.dev/: Um alias que automaticamente redireciona para o domínio principal, configurado pela padronização do TLD (third-level domain) www como o início de URLs confiáveis na internet. Também bloqueado para indexação, existe para evitar confusão.
- app.liquamonitor.dev/: Um alias destacado para a futura aplicação web do Sistema de Monitoramento de Consumo Liqua Monitor, uma interface para a API assim como a aplicação mobile.

Esse sistema foi configurado de forma a minimizar *downtime* na hospedagem com Docker, permitindo utilizar o Caddy para gerar automaticamente os certificados HTTPS, de entidades confiáveis como a Let's Encrypt, e permitir atualizações tanto em ambiente de desenvolvimento quanto produção com confiabilidade.

Os demais serviços de infraestrutura, PostgreSQL e Redis também são instanciados na forma de contêineres Docker e seguem princípios similares, porém exigem menos configuração devido à sua natureza estática.

7. Considerações Finais

A API Liqua Monitor é uma aplicação robusta para produção, que utiliza FastAPI e Caddy tanto para facilitar desenvolvimento quanto para permitir fluxos de dados velozes e previsíveis. A arquitetura modularizada permite manutenção e código autoexplicativo, essencial para a rotação de membros do time, e o fluxo de *deploy* é

simples e compreensível, com configurações básicas e amplamente disponíveis na internet.

Esse projeto, agora muito próximo do fim de seu desenvolvimento, permanecerá como um projeto educacional valioso e uma experiência real da equipe com ambiente e infraestrutura de produção. Além disso, é possível que parte da equipe do sistema se responsabilize por sua manutenção a longo prazo, aproveitando a estrutura existente.